

13주차 부품을 이어 붙이면 어디가 터지는가

13주차 · 공공 AI 통합 사례

이 장의 수치는 모두 `practice/chapter13/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

다만 이 장은 두 종류의 증거를 섞어 쓴다. 회계·집계·지연 채점은 지금 실행해 만든 `practice/chapter13/results/` 의 로그에서 인용했고, **Kafka-Docker가 있어야 나오는 값(스모크 응답, 장애 드릴, 인수 검수표)**은 과거 실행이 남긴 산출물에서 옮긴 값이다. 뒤쪽 수치의 출처는 `practice/chapter13/data/output/` 이다.

다른 주차의 산출물을 가져다 쓴 곳에는 그 주차의 `practice/chapter{N}/` 경로를 함께 적었다.

□ 1주차부터 12주차까지 데이터·모델 운영 부품을 하나씩 만들었음 ○ 수집·정제·집계·피처·훈련·배포·감시가 각각 자기 장 안에서는 작동했음 □ 이 장은 그 부품을 하나의 운영 흐름으로 다시 조립해 작은 시스템을 돌림 ○ 민원이 들어오고, 하루가 마감되면 집계가 확정되며, 그 확정치가 다음 날 예측의 입력이 되는 시스템임 □ 새 기술은 하나도 나오지 않음 ○ 대신 부품과 부품이 만나는 자리에서만 생기는 고장을 일부러 일으켜 봄 □ 이 장도 설명과 실습을 나누지 않음 ○ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐

1. 오늘의 질문

□ 부품을 옮겨 붙이는 동안 값이 변하지 않았다는 것을 무엇으로 증명할까? □ 재전송과 중복 접수로 열한 건이 더 들어왔는데 집계가 그대로라면, 그 사실을 어떻게 보일까? □ 모델 API가 죽은 순간 하루 마감은 어떻게 되어야 할까? □ 앞 주차들이 각자 옮겨 정한 값을 한 시스템에 모으면 왜 서로 부딪칠까?

2. 이 장에서 실행할 것

□ 부품을 따로 시험한 기록이 있어도 조립한 차를 그대로 출고하지는 않음 ○ 조립한 뒤 다시 재고, 일부러 급제동을 걸어 보고, 항목마다 합격 여부를 적은 검사표를 붙여 내보냄 ○ 이 장이 하는 일의 단계가 여기에 하나씩 대응함

완성차 출고 검사	이 장의 통합 시스템
부품마다 따로 남겨 둔 규격 시험 기록	6·9·10주차 실습이 각각 커밋해 둔 실측값
조립한 뒤 부품 번호를 원장과 다시 대조	서빙 앱 해시(hash)-피쳐 지문-확정치 재대조
급제동·충돌 시험을 일부러 한다	드릴(drill) 3종 — 오염·중복·의존 서비스 중단
항목마다 합격 여부를 적은 출고 검사표	인수 검수표 16항목(모두 자동 계산)

표 13.1 완성차 출고 검사와 통합 검수의 대응

- ☐ 비유가 깨지는 지점이 하나 있음 ○ 자동차 부품은 눈으로 보면 같은 부품인지 알지만, 복사된 파일은 이름이 같아도 내용이 다를 수 있음 ○ 그래서 이 장은 "같은 코드로 만들었다"가 아니라 **바이트가 같다**를 해시로 확인함 ○ 그 확인이 3절에서 숫자로 나타남
- ☐ 실행하기 전에 네 가지를 먼저 적음 ○ 산출물을 먼저 열면 거기 적힌 값이 정답처럼 보여 "왜 이 값인가"를 묻지 않게 됨

표 13.2 예측표 — 실행하기 전에 채운다

예측할 것	내 예측	실제	어긋났다면 원인
같은 민원 10건을 한 번 더 보내면 그날 확정 집계가 커지는가			
JSON으로 읽히지 않는 이벤트 1건이 들어오면 그날 마감이 실패하는가			
마감 직전에 모델 API를 정지시키면 그날 집계가 확정되는가			
파일을 읽는 배치와 스트림이 같은 날의 확정치를 같게 내는가			

- ☐ 이 장에서 실행할 코드는 둘이고 하나는 선택임

순서	실행할 파일	하는 일	절	선행 조건
1	<code>code/13-2-replay-accounting.py</code>	같은 입력-같은 규칙을 표준 라이브러리로 재생해 회계-집계-지연 채점을 다시 계산	3·4·6	Python만
선택	<code>run_chapter13.py</code>	통합 MVP를 컨테이너로 띄워 3일 운영과 드릴 3종을 완주	3·4·5·6	Docker 데몬

- 1번은 Kafka도 Docker도 쓰지 않으므로 강의실에서 그대로 돌아감 ○ 2번은 Docker 데몬이 필요하므로, 켤 수 없으면 커밋된 산출물을 열어 값을 읽음

시작 전 준비

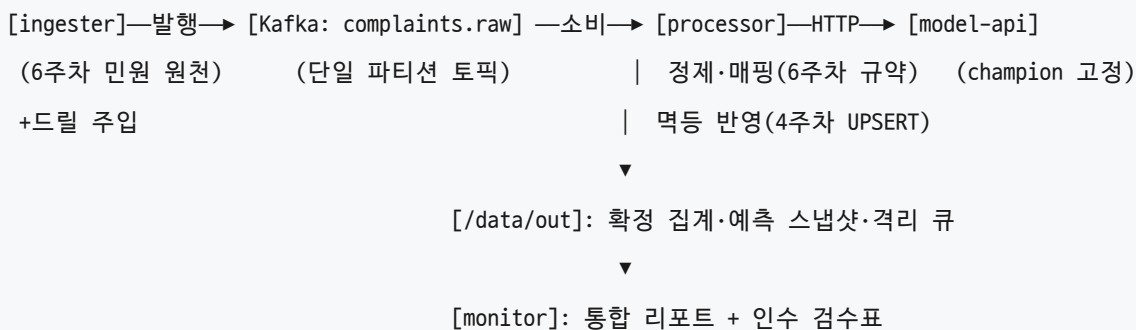
- ☐ 선택 실습인 `run_chapter13.py` 는 **Docker Desktop**이 실행 중이어야 함 ○ 통합 MVP를 컨테이너 네 개로 띄워 드릴을 돌리기 때문임

- Docker를 켤 수 없어도 이 장은 끝까지 따라갈 수 있다.
- 1번 실습이 회계·집계·지연 채점을 다시 계산해 주고, 컨테이너가 있어야 나오는 값은 `practice/chapter13/data/output/` 에 이미 커밋되어 있다.
- 다만 커밋된 산출물의 값을 읽을 때는 그것이 과거 실행의 기록이라는 점을 잊지 않는다.

3. 재조립은 말이 아니라 값으로 증명한다

□ 통합에서 가장 흔한 의문은 "옳게 붙이다 무엇인가 달라지지 않았나"임 ○ 이 의문에 말로 답할 수 없는 이유는 부품이 코드가 아니라 **복사된 파일**이기 때문임 ○ 같은 이름의 파일이 한 글자 달라도 시스템은 아무 말 없이 돌아감 □ 그래서 이 실습은 앞 주차가 커밋해 둔 실측과 이 주차의 실행 결과를 코드가 자동 대조함 ○ 대조 결과가 인수 검수표의 항목이 됨

□ 통합 MVP(Minimum Viable Product, 핵심 경로만 넣은 최소 운영 시스템)의 구조는 다음과 같음 ○ 모든 연결은 Docker Compose 내부 네트워크 안에 있고, 바깥에 열리는 것은 모델 API 포트 하나뿐임



□ 부품마다 원래 주차가 있음 ○ 메시지 버스는 4주차의 Kafka임 ○ 중복 방지는 4주차의 `complaint_id` 업서트(UPSERT)임 ○ 정제·표준코드 매핑·`_SUCCESS` 마커(marker)는 6주차의 규약임 ○ 모델은 9주차 승격한 champion이고, 서버는 10주차 `app.py` 를 바이트 그대로 복사해 씀 ○ 지연 채점은 11주차의 감시 경로임 □ 13주차가 새로 만든 규칙은 하나도 없음

□ 하루의 흐름은 두 개의 시계로 돌아감 ○ 접수는 스트림(stream)의 시계를 따라 이벤트가 오는 대로 정제·반영됨 ○ 확정과 예측은 배치의 시계를 따라 하루가 마감된 뒤에야 나감 □ 마감이 오면 processor가 하는 일이 셋임 ○ 그날의 집계를 확정함 ○ 어제 만든 예측을 오늘 도착한 실제 건수로 채점함 ○ 오늘 확정 건수를 입력으로 내일 예측을 만들 ○ **확정 → 채점 → 예측**이라는 이 세 박자가 매일 반복됨

□ 첫 실습은 이 세 박자를 Kafka 없이 다시 돌려 6주차 배치가 낸 확정치와 맞춰 봄

```
py practice/chapter13/code/13-2-replay-accounting.py
```

```
# 출처: practice/chapter13/results/13-2-replay-accounting.log
[replay] 2026-07-01 마감: 물리수신 20 / 고유 20 (중복 흡수 0) → 매핑 18 · 미기재 1 · 미매핑 1 · 보존 True
[replay] 2026-07-02 마감: 물리수신 28 / 고유 17 (중복 흡수 11) → 매핑 16 · 미기재 0 · 미매핑 1 · 보존 True
[replay] 2026-07-03 마감: 물리수신 22 / 고유 22 (중복 흡수 0) → 매핑 20 · 미기재 1 · 미매핑 1 · 보존 True
[PASS] 2026-07-01_확정집계가_6주차_배치와_일치
[PASS] 2026-07-02_확정집계가_6주차_배치와_일치
[PASS] 2026-07-03_확정집계가_6주차_배치와_일치
CH13_REPLAY_PASS
```

```
# 출처: practice/chapter6/data/output/daily/2026-07-03/daily_summary.json
"by_region": [ {"count": 6, "lawd_cd": "11440", "region": "마포구"},
                {"count": 5, "lawd_cd": "11620", "region": "관악구"},
                {"count": 9, "lawd_cd": "11680", "region": "강남구"} ],
"mapped_total": 20
```

- 화면에 나온 값에서 읽을 것 ○ **경로가 다른데 확정치가 같음** — 6주차는 파일을 읽는 배치로 집계했고 이 실습은 이벤트를 하나씩 소비하며 집계했는데, 7/3의 6·5·9가 그대로 다시 나왔음 ○ **같아야 한다는 주장이 아니라 대조 결과임** — [PASS] 줄은 코드가 두 파일을 열어 비교한 결과이고, 사람이 "맞습니다"라고 적은 칸이 아님 ○ **마지막 줄이 CH13_REPLAY_PASS 임** — 열네 개 대조가 모두 통과했다는 뜻이고, 이 줄이 없으면 위 수치를 읽지 않음
- 여기서 얻는 결론이 이 절의 핵심임 ○ **처리 방식이 바뀌어도 정제 규칙·역등 키·집계 기준이라는 업무 규약이 같으면 결과가 같다.** ○ 배치냐 스트림이냐는 언제 계산하는가의 문제이지 무엇을 세는가의 문제가 아님
- 부품의 신원을 확인하는 항목은 값 대조가 아니라 지문 대조임 ○ 컨테이너로 완주한 실행은 두 지문을 함께 확인해 두었음

```
# 출처: practice/chapter13/data/output/ch13_integration_report.json
"reassembly_checks": {
  "app_sha256_matches_ch10": true,
  "feature_fingerprint_matches_ch9": true,
  "aggregation_matches_ch7": {"2026-07-01": true, "2026-07-02": true, "2026-07-03": true},
  "all_forecasts_equal_ch9_champion_6_0": true,
  "overall_delayed_mae_equals_ch9_train_mae": true }
```

```
# 출처: practice/chapter13/data/output/ch13_run_log_preserved.txt
[bootstrap] 훈련 쌍 6개, 지문 96f6b6b3ce9dcdd1...
서빙 앱 재조립 확인: sha256 일치 (ec7ec65397a3194a...)
```

□ 화면에 나온 값에서 읽을 것 ○ 피쳐 지문이 9주차 기록과 같음 — 96f6b6b3... 은 9주차가 훈련에 쓴 데이터의 지문이고(practice/chapter9/data/output/ch9_experiment_report.json), 이 장이 재구성한 훈련 쌍의 지문이 같은 값으로 나왔음 ○ 서빙 앱은 값이 아니라 바이트로 확인함 — 예측이 같게 나오는 것만으로는 같은 앱이라고 말할 수 없으므로 파일 자체의 sha256을 맞춰 봄 ○ 전 호출이 6.0임 — 9주차 champion은 학습 자료의 평균을 그대로 답하는 평균 예측기라 입력이 달라도 같은 값을 냄

- 해시는 그 실행 시점의 값이다.
- 10주차 app.py 가 한 글자라도 바뀌면 ec7ec653... 은 다른 값이 되고, 대조는 일치 에서 불일치! 로 바뀐다.
- 그래서 해시를 본문에 외워 두는 것이 아니라 매 실행에서 다시 계산하도록 코드에 넣어 둔다.

4. 중복은 지워지는 것이 아니라 흡수된다

□ 메시지 전송은 대개 at-least-once로 설계됨 ○ 보낸 쪽이 확인 응답을 받지 못하면 다시 보냄 ○ 유실을 막는 대신 중복이 생김 □ 여기에 원천 자체의 중복도 섞임 ○ 민원인이 같은 건을 두 번 접수하면 재전송이 아닌데도 같은 식별자가 두 번 도착함 □ 두 중복은 원인이 다르지만 소비자 쪽에서 보이는 모습이 똑같은 ○ 같은 complaint_id 가 다시 도착한 것임

□ 드릴 D2는 이 상황을 그대로 재현함 ○ 7/2의 이벤트 18건을 발행한 뒤 마지막 10건을 한 번 더 발행했음 ○ 앞 절의 실행에서 이 드릴이 이미 돌아갔으므로 같은 로그의 7/2 줄을 다시 봄

```
# 출처: practice/chapter13/results/13-2-replay-accounting.log
[replay] 2026-07-02 마감: 물리수신 28 / 고유 17 (중복 흡수 11) → 매핑 16 · 미기재 0 · 미매핑 1 · 보존 True
[PASS] 2026-07-02_정제회계가_6주차_배치와_일치
[PASS] 2026-07-02_회계보존식_성립
```

```
# 출처: practice/chapter6/data/output/daily/2026-07-02/quality.json
{"physical": 18, "dup_removed": 1, "valid": 17,
 "mapped": 16, "missing": 0, "unmapped": 1}
```

□ 화면에 나온 값에서 읽을 것 ○ 두 파일의 물리 수신이 다름 — 6주차 배치는 파일을 읽었으므로 18건이고, 이 장은 재전송 10건이 더 왔으므로 28건임 ○ 고유는 둘 다 17건임 — 6주차가 흡수한 중복 1건은 원천 파일 안의 중복 접수이고, 이 장이 흡수한 11건은 그 1건에 수송 재전송 10건을 더한 것임 ○ 매핑 16건이 변하지 않음 — 물리 수신이 10건 늘어나도 확정 집계는 그대로임 ○ 보존 True 가 함께 찍힘 — 늘어난 건수가 어디로 갔는지 등식으로 설명된다는 뜻임

□ 4주차의 멍등(idempotent) 키가 강력한 이유가 여기 있음 ○ 업무 식별자를 키로 삼아 업서트하면 중복의 원인을 구분할 필요 없이 둘 다 무해해짐 ○ 재수신은 업무 상태를 바꾸지 않고 관찰 횟수만 올린 흔적으로 남음 ○ 두 계층에 각각 다른 중복 제거 장치를 두었다면, 어느 쪽이 어느 중복을 맡는지부터 정해야 했을 것임 □ 같은 원칙이 소비자의 반대편에도 걸려 있음 ○ processor는 오프셋(offset) 자동 커밋을 끄고, 메시지를 파일과 상태 DB에 다 쓴 뒤에야 커밋함 ○ 자동 커밋은 처리가 끝나기 전에 오프셋을 밀어 올릴 수 있어, 그 사이에 멈추면 그 메시지가 재생되지 않음 ○ 늦게 커밋하면 재생이 생기지만, 그 재생을 무해하게 만드는 것이 위의 멍등 설계임 □ 전달 보증과 멍등은 별개 기능이 아니라 한 쌍임

회계 보존식이 하는 일

□ "집계가 변하지 않았다"는 말만으로는 부족함 ○ 중복이 흡수된 것인지 애초에 없었던 것인지 구분되지 않기 때문임 ○ 그래서 마감마다 두 개의 등식을 기록함 □ 7/2에 기록된 등식은 다음 둘임 ○ 물리 28 = 고유 17 + 흡수 11 ○ 고유 17 = 매핑 16 + 미기재 0 + 미매핑 1 □ 왼쪽과 오른쪽이 맞아떨어지면 사라진 건이 없다는 뜻이고, 어긋나면 어느 갈래에서 조용히 없어졌다는 뜻임 ○ 이 등식이 로그의 보존 True 이고, 컨테이너 실행에서는 검수표 04항임

□ 여기서 배울 것은 "중복이 없었다"가 아님 ○ 중복이 분명히 있었는데도 값이 같다는 것을 회계로 증명할 수 있다는 점임

5. 실패는 가장 작은 단위로 잘라 값으로 남긴다

□ 앞의 두 절은 정상 경로였고, 이 절은 부품이 실제로 망가지는 자리임 ○ 여기부터 쓰는 값은 컨테이너로 완주한 과거 실행이 남긴 기록임

D1 — 오염 이벤트 한 건이 전체를 멈추지 않는다

□ 7/3의 정상 이벤트 사이에 JSON으로 읽히지 않는 페이로드 1건을 끼워 발행했음 ○ 맨 앞이나 맨 뒤가 아니라 열 번째 뒤에 넣은 이유가 있음 ○ 파이프라인이 "그 자리에서" 멈추지 않고 다음 메시지로 계속 가는지를 봐야 하기 때문임 □ processor는 그 메시지를 격리 큐(quarantine queue) 파일에 원문과 사유재 보존하고 넘어갔음

```
cd practice/chapter13 && python run_chapter13.py # Docker 데몬이 필요한 선택 실습
```

```
# 출처: practice/chapter13/data/output/ch13_run_log_preserved.txt
```

```
[processor] 오염 이벤트 격리(json_parse_error) - 파이프라인은 계속
```

```
[processor] 2026-07-03 마감: 물리수신 22 / 고유 22 (중복 흡수 0) → 매핑 20 · 미기재 1 · 미매핑 1 · 보존 True
```

□ 화면에 나온 값에서 읽을 것 ○ **격리 다음 줄이 마감임** — 오염 이벤트 뒤의 이벤트들이 계속 처리되어 그날 마감까지 도달했음 ○ **매핑 20건이 온전함** — 오염 1건은 집계 밖의 파일로 빠졌으므로 확정치에 섞이지 않았음 ○ **사유가 값으로 남음** — `json_parse_error` 라는 분류가 남아야 나중에 상류 버그인지 공격인지 가려낼 수 있음

□ 격리 큐가 없으면 선택지는 둘뿐임 ○ 파이프라인이 그 자리에서 멈추거나, 오염이 집계에 섞여 들어가거나임 ○ 앞은 가용성을 잃고 뒤는 정합성을 잃음 □ 2주차에서 세운 원칙이 셋째 길임 ○ **잘못된 데이터는 버리지도 통과시키지도 않고 분리해서 보존한다.**

D3 — 마감은 살고 예측만 대기한다

□ 가장 중요한 드릴임 ○ 7/3 마감 직전에 모델 API 컨테이너(container)를 정지시키고 그대로 마감을 진행했음

```
# 출처: practice/chapter13/data/output/ch13_run_log_preserved.txt
model-api 중단됨 - 이 상태로 7/3 마감을 진행한다
[processor] 2026-07-03: 지연 레이블 도착 - 예측 3건 채점, 일 MAE 1.3333
[processor] 예측 2026-07-04 마포구: 실패(api_unreachable) → pending_retry
[processor] 2026-07-03: _SUCCESS 확정
마감 확정(_SUCCESS) + 예측 대기 3건 (오류: ['api_unreachable'])
복구 수렴: 대기 0건, ok 9건
```

```
# 출처: practice/chapter13/data/output/ch13_integration_report.json
"D3_api_outage_recovery": {
  "day3_closed_despite_outage": true, "pending_at_close": 3,
  "pending_errors": ["api_unreachable"],
  "pending_after_recovery": 0, "recovered_attempts": [2] }
```

□ 화면에 나온 값에서 읽을 것 ○ **모델이 없어도 되는 일은 전부 끝났음** — 집계 확정과 어제 예측의 채점은 모델 호출이 필요 없으므로 중단 중에도 완료되었음 ○ **실패가 예측 3건으로 잘렸음** — 마감 전체가 아니라 모델이 필요한 부분만 `pending_retry`로 남았고, 사유와 입력값이 함께 기록되었음 ○ **_SUCCESS가 찍혔음** — 그날 집계는 확정본이며, 장애와 무관하게 다음 날 절차가 이 값을 믿고 쓸 수 있음 ○ **복구가 2회째 시도에서 수렴했음** — `recovered_attempts`가 [2]라는 것은 세 건 모두 재시도 한 번으로 끝났다는 뜻임

□ 이것이 **부분 실패 격리**임 ○ "모델 API가 죽으면 마감 전체가 실패"가 아님 ○ 실패를 가장 작은 단위로 잘라 상태로 기록하고 나머지는 확정함 □ 재시도가 안전했던 이유는 복구 코드가 아니라 평시의 설계에 있음 ○ 예측 기록이 (대상일, 지역)을 키로 하는 멍든 쓰기라서, 재시도가 겹쳐도 중복 기록이 생기지 않음

격리해 두면 새벽 2시가 달라진다

□ 이 설계가 무엇을 했는지는 장애가 난 뒤에 드러남 ○ 리포트에 대기 3건이 찍혀 있을 때 당직자가 할 일은 원인을 추측하는 것이 아니라 `/health` 를 한 번 호출해 보는 것임 ○ 응답이 없으면 컨테이너를 다시 띄우고 재시도를 발행한 뒤 대기가 0건이 되는지 보면 끝남 □ 더 중요한 것은 **확인하지 않아도 되는 것이** 미리 정해져 있다는 점임 ○ 오늘 집계가 손상되었는지는 확인할 필요가 없음 — `_SUCCESS` 가 있으므로 확정본임 ○ 예측이 유실되었는지도 확인할 필요가 없음 — 대기 3건이 입력값째 남아 있음 ○ 격리가 없었다면 이 두 가지부터 새벽에 뒤져야 했을 것임

□ 세 드릴을 관통하는 질문은 하나임 — **실패를 어디에 둘 것인가.** ○ 오염은 격리 큐에 둬 ○ 중복은 맥등 키에 둬 ○ 의존 서비스의 중단은 `pending_retry` 상태에 둬 □ 셋 다 실패를 없애지 않고 값으로 남김 ○ 값으로 남아 있어야 나중에 세고 감사할 수 있음 ○ 격리 큐에 쌓인 건수가 늘면 상류의 형식이 바뀔 것이고, 그 건들이 오래 머물면 재처리 절차가 돌지 않는 것임

표 13.3 3일 운영 시뮬레이션 실측 (출처: `practice/chapter13/data/output/ch13_integration_report.json`, `ch13_run_log_preserved.txt`)

항목	7/1	7/2 (드릴 D2)	7/3 (드릴 D1·D3)
물리 수신	20	28	22
고유(맥등 흡수 후)	20	17	22
중복 흡수	0	11(재전송 10 + 중복 접수 1)	0
매핑/미기재/미매핑	18/1/1	16/0/1	20/1/1
확정 집계(마포·관악·강남)	5·5·8	5·5·6	6·5·9
익일 예측(3개 구)	전부 6.0	전부 6.0	중단 → 대기 3건 → 복구 후 6.0
전일 예측의 채점(일 MAE)	—	0.6667	1.3333
격리(오염)	0	0	1

6. 계층마다 웅았던 값이 한 시스템에서 부딪친다

□ 통합에서 실제로 터지는 것은 부품의 결함이 아니라 계층별 결정의 불일치임 □ 앞 주차들은 각자의 맥락에서 값을 하나씩 정했음 ○ 4주차는 파티션(partition) 수와 중복 제거 키를, 6주차는 배치 주기와 마커 규약을 정함 ○ 9주차는 승격 게이트를, 11주차는 감시 임계와 판정 주기를 정함 □ 각 값은 자기 주차 안에서 근거가 분명했는데, 한 시스템에 모으면 서로를 무효로 만드는 조합이 나옴

□ **파티션 수 대 마감 순서** ○ 4주차는 처리량을 위해 파티션을 여럿 두는 구성을 다뤘음 ○ 6주차의 일 마감은 그날 이벤트가 전부 도착한 뒤에 마감 신호가 온다는 것을 전제함 ○ 파티션이 여럿이면 그 전제가 깨지므로 이 MVP는 파티션을 1개로 고정했음 – 처리량을 마감 순서와 맞바꾼 것이고, 그 맞바꿈을 설계에

기록해 둬

□ **게이트 값 대 표본 크기** ○ 9주차의 승격 게이트는 MAE 1.05이고, 11주차 방식의 지연 채점은 하루 표본이 3건임 ○ 표본 3건짜리 MAE로 게이트를 판정하면 하루의 우연이 재학습을 부름 ○ 앞 절의 실습이 이 충돌을 숫자로 만들어 냄

```
# 출처: practice/chapter13/results/13-2-replay-accounting.log
[replay] 2026-07-02: 지연 레이블 도착 - 예측 3건 채점, 일 MAE 0.6667
[replay] 2026-07-03: 지연 레이블 도착 - 예측 3건 채점, 일 MAE 1.3333
== 지연 평가: 일별 MAE {'2026-07-02': 0.6667, '2026-07-03': 1.3333}, 전체 1.0, 미채점 3건 ==
[PASS] 지연MAE가_통합실행과_일치
```

```
# 출처: practice/chapter13/data/output/ch13_integration_report.json
"delayed_evaluation": {
  "labeled": 6, "mae_by_target_date": {"2026-07-02": 0.6667, "2026-07-03": 1.3333},
  "overall_mae": 1.0, "mae_gate": 1.05,
  "gate_exceeded_days": ["2026-07-03"],
  "pending_labels": ["2026-07-04", "2026-07-04", "2026-07-04"] }
```

□ 화면에 나온 값에서 읽을 것 ○ **하루치로 보면 게이트를 넘고 전체로 보면 넘지 않음** — 7/3의 1.3333은 게이트 1.05를 넘지만 6건 전체의 MAE는 1.0임 ○ **하루 사이의 출렁임이 0.6667과 1.3333임** — 모델은 그대로이고 표본이 3건이라 이 폭이 나온 것이므로, 이 값으로 재학습을 결재하면 우연에 반응하는 것임 ○ **전체 1.0이 9주차 훈련 MAE와 같음** — 채점된 6건은 9주차가 학습에 쓴 쌍과 같은 데이터이고 모델도 같으므로 계산상 같아야 하고, 그 일치가 감시 경로가 제대로 계산하고 있다는 확인이 됨 ○ **미채점 3건이 남아 있음** — 7/4의 실제 건수가 아직 도착하지 않았으므로, 지연 채점은 언제나 채점된 것과 기다리는 것을 함께 보고함

□ 이 MVP는 게이트 값을 바꾸는 대신 판정 주기를 늘리는 쪽을 택했음 ○ 하루치 MAE는 재학습 결재가 아니라 감시 강화 신호로만 읽음

□ **두 층의 맥등 키** ○ 4주차의 맥등 키는 `complaint_id` 이고, 이 장의 예측 재시도 키는 (대상일, 지역)임 ○ 서로 다른 층의 결정인데 둘 다 맥등이라서 D3의 재시도가 안전했음 — 한쪽만 맥등이 아니었어도 복구 절차 자체가 새 오염을 만들었을 것임

□ 세 조합의 형태는 같음 — **한 계층의 최적값이 다른 계층의 전제를 깬다.** ○ 그래서 통합 설계의 첫 작업은 새 값을 정하는 것이 아니라, 이미 정해진 값을 한 표에 모아 충돌을 찾는 것임 □ 충돌이 나오지 않는 칸도 있음 ○ 12주차의 접근 등급이 그러함 — 이 MVP는 피처가 한 종이고 그 피처를 쓰는 모델도 하나여서 접근 규약이 암묵으로 남았기 때문임 ○ 그래도 그 칸을 비우지 않고 "왜 충돌이 없는지"를 적음 — 충돌이 없다는 판단에도 조건이 붙어 있고, 피처가 늘어나는 순간 그 조건이 사라지기 때문임 □ 이 점검은 요구가 바뀔 때마다 다시 해야 함 ○ 같은 부품으로 재난 대응을 만든다면 침수 신고 집계가 하루 마감

기다릴 수 없으므로 5주차의 분 단위 창과 워터마크가 들어옴 ○ 그 순간 "일 마감"이 그날 이벤트를 전부 받은 뒤에 온다"는 전제 위에 서 있던 단일 파티션 결정이 근거를 잃음 ○ 부품은 그대로인데 시계가 바뀌면 조합이 통째로 달라짐

□ 이 표를 만드는 일이 왜 문서 작업이 아닌지도 여기서 분명해짐 ○ 파티션 수와 게이트 값은 실행 로그 어디에도 "이 둘이 서로 부딪칩니다"라고 찍히지 않음 ○ 로그는 그날 마감"이 되었는지, MAE가 얼마였는지만 말해 줌 ○ 그 값이 다른 계층의 전제를 깨고 있는지는 사람이 표를 만들어 맞대어 봐야 보임

□ 마지막으로 인수 검수표를 확인함 ○ 컨테이너로 완주한 실행은 16항목을 자동 계산해 정본 파일로 남겼음

```
# 출처: practice/chapter13/data/output/ch13_acceptance_checklist.json
```

```
"04_회계_보존식_성립(물리=고유+중복,고유=매핑+미기재+미매핑)": true,
```

```
"05_중복_재전송_무해(물리28→매핑16_불변)": true,
```

```
"10_부분_실패_격리(중단_중_마감_확정+예측만_대기)": true,
```

```
"13_지연_MAE가_9장_훈련_MAE와_일치(1.0_손검산)": true,
```

```
"14_재조립_증명_서빙앱_바이트동일(10장_sha256)": true
```

□ 화면에 나온 값에서 읽을 것 ○ 항목 이름에 판정 근거가 들어 있음 — "잘 되었다"가 아니라 어떤 등식. 어떤 값과 맞췄는지가 이름에 적혀 있음 ○ 전부 계산된 참·거짓임 — 사람이 "확인했습니다"라고 적을 자리가 없음 ○ 예측 자체는 아무것도 집행하지 않음 — 예측은 기록으로만 남고, 자원을 옮길지 안내를 보낼지는 사람이 판단함

7. 핵심 요약

□ 1. 재조립이 원본과 같은 값을 낸다는 것을 값으로 증명한다. ○ 배치로 집계한 6주차와 이벤트를 소비해 집계한 이 주차가 3일치 확정치를 똑같이 냈고, 그 대조를 코드가 [PASS] 로 출력함 ○ 처리 방식이 아니라 정제·먹등·집계라는 업무 규약이 결과를 정한다는 뜻임 ○ 부품의 신원은 값이 아니라 지문으로 확인하며, 피쳐 지문과 서빙 앱 바이트를 앞 주차 기록과 맞춤(3절)

□ 2. 중복은 지워지는 것이 아니라 흡수되며, 흡수는 회계로 증명한다. ○ 7/2에 물리 28건이 들어와 고유 17건이 되고 매핑 16건으로 확정되었으며, 6주차 배치의 매핑 16건과 같았음 ○ 흡수된 11건은 수송 재전송 10건과 원천의 중복 접수 1건이고, 원인이 달라도 먹등 키 하나가 둘 다 무해하게 만듦 ○ 물리=고유+흡수, 고유=매핑+미기재+미매핑이라는 두 등식이 성립해야 사라진 건이 없다고 말할 수 있음(4절)

□ 3. 실패는 가장 작은 단위로 잘라 값으로 남긴다. ○ 오염 1건은 격리 큐로 빠지고 그날 매핑 20건은 온전했으며 마감도 확정되었음 ○ 모델 API가 죽은 7/3에도 집계는 확정되고 어제 예측은 채점되었으며, 모델이 필요한 3건만 `pending_retry` 로 남았다가 2회째 시도에서 0건으로 수렴했음 ○ 재시도가 안전했던 것은 복구 코드가 아니라 (대상일, 지역) 먹등 쓰기 덕분임(5절)

□ 4. 통합 설계의 첫 작업은 값을 정하는 것이 아니라 충돌을 찾는 것이다. ○ 파티션을 여럿 두면 일마감의 전제가 깨지므로 이 MVP는 1개로 고정했음 ○ 하루 표본 3건의 MAE가 0.6667과 1.3333으로 출력였고 7/3은 게이트 1.05를 넘었지만, 6건 전체 MAE는 1.0이었음 ○ 각 값이 자기 주차 안에서 옳았다는 사실이 한 시스템 안에서도 옳다는 것을 보장하지 않음(6절)

8. 과제

□ `py practice/chapter13/code/13-2-replay-accounting.py` 를 실행해 만든

`practice/chapter13/data/output/ch13_replay_accounting.json` 을 LMS에 업로드함 □ 함께 계층 결정 일관성 점검표를 만들어 올림 ○ 행은 일곱 개임 — 토평·파티션 수(4주차), 중복 제거 키(4주차), 창·마감 방식(5-6주차), 배치 주기(6주차), 승격 게이트(9주차), 감시 임계·판정 주기(11주차), 접근 등급(12주차) ○ 열은 네 개임 — 원래 주차가 정한 값, 이 통합에서 쓴 값, 왜 그 값인가, 어느 결정과 충돌할 수 있는가 ○ 넷째 열에 "앞 주차에서 그렇게 정했으니까"를 쓰지 않고 다른 결정과 어떻게 맞물리는지를 적음 ○ 마지막 열은 한 칸도 비우지 않으며, 충돌 후보가 없다고 보면 왜 없는지를 적음

□ 이어서 세 가지를 각각 세 문장 이내로 씀

1. 본인이 만든 `ch13_replay_accounting.json` 에서 2026-07-02의

`physical_received` · `unique` · `dedup_dropped` · `mapped` 를 적고, 물리 수신이 열 건 늘었는데도 `mapped` 가 변하지 않은 이유를 두 등식으로 설명한다.

2. 같은 파일의 `cross_checks` 에서 6주차 배치와 일치한 항목을 찾아 값을 적고, 파일을 읽는 배치와 이벤트를 소비하는 경로가 같은 확정치를 낸 이유를 정제·먹등·집계·마감 중에서 골라 설명한다.

3. 저장소에 커밋된 `practice/chapter13/data/output/ch13_acceptance_checklist.json` 을 열어 13항과 14항의 값을 적고, 두 항목이 각각 무엇을 보증하는지 쓴다. Docker를 켤 수 있었다면 `run_chapter13.py` 를 실행해 만든 본인 파일로 답한다.

□ 점검표는 값의 정답 여부가 아니라 셋째·넷째 열이 채워져 있는지를 봄 □ 수업일 포함 7일 이내에 제출함

9. 더 알아보기

□ 이 장에서 자른 내용은 아래에서 이어 볼 수 있음 ○ **통합 MVP 전체 코드** — 발행·소비·정제·마감·예측·감시가 어떻게 네 컨테이너로 나뉘는지: `practice/chapter13/code/13-1-integrated-mvp/` . 재조립 대조의 기대값 상수는 `monitor.py` 머리에 앞 주차 출처와 함께 적혀 있음 ○ **오프라인 재생이 무엇을 못 하는가** — `practice/chapter13/code/13-2-replay-accounting.py` 의 머리 주석에 모델 API를 호출하지 않는다는 한계와, `champion` 예측값을 어느 산출물에서 읽어 오는지가 적혀 있음 ○ **드릴을 바꿔 보기** — `13-2-replay-accounting.py` 의 `DRILLS` 표에서 재전송 건수나 오염 건수를 바꾸면 회계 보존식이 어떻게 따라 움직이는지 볼 수 있음. 확정 집계는 변하지 않는 범위가 어디까지인지 확인함 ○ **컨테이너 이미지와 버전 고정** — 어떤 이미지와 어떤 패키지 버전으로 돌렸는지: `practice/chapter13/data/output/ch13_integration_report.json` 의

- versions ○ 멱등성의 정의와 재시도 규약 — <https://datatracker.ietf.org/doc/html/rfc9110#name-idempotent-methods> ○ 검수를 검증 가능한 항목으로 명세하는 접근 — <https://research.google/pubs/pub46555/>
- 생성형 모델을 같은 방식으로 운영할 때 무엇이 달라지는가 — 14주차에서 다룸